GH-300

Code Health

with GitHub Copilot

Tests, debugging, and refactoring — the Copilot commands, agents, and workflows that keep a codebase healthy.

/tests | /explain · /fix · /optimize | Debugger Agent | Custom Agents | Cloud Agent

Sandeep Soni · Deccansoft Software Services Pvt Ltd

Deck 8 · GH-300 Certified GitHub Copilot Expert Training

Part 1 · Testing

Generating, extending, and maintaining tests with Copilot.

Why generate tests with Copilot?

Faster scaffolding

AAA structure, mock setup, parameterized cases — generated in seconds. You spend your time on the cases only you can think of.

Coverage you actually write

Tests are the most-skipped task in tired projects. When the cost drops to a prompt, coverage stops being aspirational.

Better when paired with agents

Plan mode designs the suite, agent mode runs and iterates, Cloud Coding Agent can open the test PR. You review.

Honest framing — what we will NOT claim today

No magic numbers. Productivity gains depend on the codebase. Greenfield TypeScript will see different gains than a 15-year-old C# monolith.
Generated ≠ correct. Every Copilot-generated test must be read, ran, and fixed before it is trustworthy. Failing-but-meaningful tests > passing-but-shallow.
AI is the scaffold; you are the architect. Edge cases that only domain experts know remain your job. Copilot will get the obvious 70% — you add the 30% that matters.

Five Copilot modes — match the mode to the test task

Ask

"Why is this test flaky?" No file changes.

When

Q&A on existing tests.

Edit

/tests on a selection. Single-turn write/edit.

When

Add tests next to code.

Agent

Generates → runs → fixes failures → iterates.

When

Whole test suite from scratch.

Plan

Reviews the test plan with you before any code is written.

When

New, large feature surfaces.

Custom

.agent.md — e.g. TDD-Writer with scope locked to /tests folder.

When

Repeatable team workflows.

NEW (Apr 2026 preview): Inline Agent Mode brings full agent capabilities directly into inline chat — generate, run, and iterate on tests without leaving the editor pane. Works in VS Code and JetBrains.

The /tests command — what it sees and what it does

What Copilot sees

- The selected code (or #file target)
- Surrounding code in the same file
- Open tabs + recently-viewed files
- Detected test framework (Jest, xUnit, pytest, JUnit…)
- Project layout (test folder conventions, naming)
- .github/copilot-instructions.md and any .instructions.md scoped to test paths
- Existing tests for the same module (used as style examples)

What it returns

- Happy-path tests for visible behaviours
- Basic edge cases (null/empty/0) — usually shallow without prompting
- Mock setup for obvious dependencies
- AAA structure unless your existing tests use a different style
- Imports — but verify the paths: they are the most common error
- New files written to the conventional test path, or appended to an existing test file
- A short summary explaining what each test covers

Five /tests variations — specific prompts get specific coverage

BASIC

`/tests #selection`

Happy path + minimal edges. Use only as a starting baseline.

FRAMEWORK

`/tests #selection using xUnit / pytest / Jest`

Pin the framework explicitly when the project has more than one option.

EDGE CASES

`/tests #selection — include null, empty, 0, MAX_INT, Unicode, whitespace`

Enumerate the edge categories you care about. Generic asks → generic tests.

WITH MOCKS

`/tests #selection — mock the DB and email provider; assert calls`

Naming the deps and the assertions stops drift toward "real" calls.

FULL FILE

`/tests #file — unit + 1 integration scenario per public surface`

Best for new modules or pre-refactor coverage capture.

Strategy 1 — function-level testing

1. **Select the function**
2. **Open Copilot Chat**
3. **Run /tests #selection**
4. **Review coverage**
5. **Add domain edges**

When to use

- Utility functions and validators
- Pure business-logic methods
- Anything that takes args and returns a value (easy contract)
- A starting point when bigger tests overwhelm you

prompt

```
// In Chat
/tests #selection

// Better — front-load the requirements
/tests #selection — use Jest,
  include null / undefined / empty,
  mock the repository,
  parameterize the boundary values
```

Strategy 2 — file-level testing

1. Open the source file
2. /tests #file
3. Generated suite
4. Organize / split
5. Add integration tests

When to use

- New module without coverage
- Legacy file you are about to refactor
- Modules with multiple public surfaces
- Onboarding handover — coverage as documentation

prompt

```
// In Chat (Agent mode recommended)
/tests #file:OrderService.cs

// Then ask Copilot to split into
// describe / class-per-public-surface
// and add 1 integration test per workflow

// Run -> fix any imports -> commit
```

Strategy 3 — TDD with Copilot

1

Sign + docstring

Write the function signature and the doc/spec that describes the behaviour. Copilot reads the doc as truth.

2

Generate tests

/tests #selection — produces tests from the doc, not the (still empty) body.

3

Tests fail

Expected. Red means we have a spec that is checkable.

4

Implement

Inline completions or agent mode. Optimize after tests go green, not before.

5

Iterate to green

Re-run until all tests pass. Add cases Copilot missed. Refactor with the safety net in place.

Why this works with AI: tests are written from intent (your spec), not from the implementation Copilot was about to invent. You stay in charge of behaviour; the AI just writes the keystrokes.

Effective prompts for edge-case coverage

Vague — generic in, generic out

```prompt
/tests #selection

/tests for #file
```

Effective — explicit, enumerated, structured

```prompt
/tests #selection — include null,
  undefined, empty strings, MAX_INT,
  Unicode, and timezone boundaries
```

Name the framework

"…using Jest / xUnit / pytest" — disambiguates polyglot projects.

Name the mocks

"…mock the repo and email provider; assert each call" — stops the slide toward real I/O.

Name the contracts

"…assert that on failure, the function returns Result.NotFound and never raises" — pins behaviour, not just shape.

Ask for parameterization

"…parameterize the boundary table; one Theory / parametrize block" — keeps the file small.

Coverage-gap detection — let Copilot find the missing tests

1. **Run coverage**
2. **Feed report to Copilot**
3. **Ask which branches are untested**
4. **Generate the missing tests**
5. **Verify the gap closed**

prompt

```
// Paste coverage summary into chat

Coverage for OrderService is 64%.
Which branches and edge paths are
untested?  Use #file:OrderService.ts
as ground truth.

For each gap, return:
 1. the file & function
 2. the missing scenario
 3. a single /tests prompt I can run
```

Why this works

Coverage tools report what is uncovered, not what to do about it. Copilot bridges that gap by reading both the report and the source.

Pair with /tests in agent mode so the generated tests are immediately run against the suite and any breakages fixed in-loop.

For CI, the same flow can be a custom agent ("Coverage-Reviewer") triggered on the nightly build.

A custom "TDD-Writer" agent — least-privilege test authoring

agent.md

```
# .github/agents/tdd-writer.agent.md
---
name: TDD-Writer
description: Writes failing tests first
tools: ['search/codebase','editFiles','read/problems']
model: ['Claude Sonnet 4.6','GPT-5.2-Codex']
handoffs:
  - label: Implement Until Green
    agent: agent
    send: false
---

# Instructions
You are the TDD-Writer.
You may edit files ONLY under
/tests, /spec, **/*Tests.cs,
**/*_test.py, **/*.test.ts
You may NEVER edit production source.
On each call: read spec -> write
failing tests -> hand off -> verify.
```

Why a custom agent for tests?

Tool scope is enforcement, not advice.

A test-writer cannot accidentally "fix" the production code to make the test pass.

Handoffs guide the workflow.

Tests → "Implement Until Green" → review. One reviewable artifact per step.

Where to put it

- .github/agents/ — workspace-level, shared with the team via Git.
- User profile — personal, follows you across repos.
- .claude/agents/ — Claude Code-compatible mirror.
- Pick the workspace folder for anything you want code-reviewed.

Big test work — Plan → CLI → Cloud handoff

Plan (local)

Plan mode designs the suite — which surfaces, which fixtures, which mocks. You approve before any test code is written.

CLI (background)

Continue in Copilot CLI. Worktree isolation. Tests are written + run + iterated without blocking your editor. Parallelisable across modules.

Cloud (PR)

/delegate to the Cloud Coding Agent. Pushes the branch, runs CI, self-fixes failures, opens a draft PR with the diff.

When to use this pattern

Use it for coverage-uplift work on legacy modules, post-incident regression test campaigns, or any test work that spans more than ~10 files. Skip the handoff for single-function unit tests — direct /tests is faster.

CLOSE

Testing — key takeaways

1

Tests are the most-skipped task in tired projects

Copilot drops the cost to a prompt — that is how it changes coverage in the real world.

2

Match the mode to the test task

Ask for Q&A · Edit for one selection · Agent for whole suites · Plan for big features · Custom for repeatable team workflows.

3

Specific prompts → specific coverage

Name the framework, name the mocks, enumerate the edge categories. Generic asks return generic tests.

4

Copilot mirrors your existing tests

It infers the framework, folder conventions, and assertion style from the tests already in the repo. Seed one good test and the rest inherit it.

5

New 2026 surfaces are worth learning

Coverage-gap prompts, flaky-test triage, TDD-Writer custom agents, and the Cloud Coding Agent for issue-to-PR test work.

6

You still own the verdict

Generated tests are a starting line. Read every assertion, run every suite, and add the edges only you know.

Part 2 · Debugging

From stack traces and failing tests to root cause — /explain, /fix, and the Debugger Agent.

Why debug with Copilot?

Reads what you skim

Copilot has no fatigue. Long stack traces, dense logs, unfamiliar libraries — it reads every line and groups what matters.

Hypotheses, not answers

Treat suggestions as candidate hypotheses, ranked. Your job is to falsify them with evidence — not to trust them on sight.

Agent loops close fast

Generate fix → run tests → read failures → propose new fix. The wall-clock cost of one more attempt drops to seconds.

Honest framing — what we will NOT claim today

Copilot does not run your code. It reasons from text. Reproducing a bug, attaching a debugger, and observing state are still your job (or the Debugger Agent's).

It hallucinates causes. The most-plausible-looking fix is sometimes wrong about which line is wrong. Verify with a test or a print before you commit.

It does not replace mental models. Use Copilot to accelerate hypothesis generation; you still own the causal chain.

Four debug surfaces — pick the right one

Inline · /explain · /fix

On a selection or an error squiggle. Fast hypothesis, quick patch.

When

Single function, obvious-ish bug.

Chat (Ask / Edit)

For multi-step reasoning over multiple files. Use #file, #problems, #terminalSelection.

When

Cross-file bug, async issue.

Agent / Plan

Agent reads, runs, iterates. Plan reviews the approach before any code changes.

When

Reproducible failing test; unclear scope.

Debugger Agent · Cloud

Visual Studio Debugger Agent (Apr 2026 GA). Cloud agent reproduces from a GitHub issue.

When

Runtime state needed; or "fix this issue" assigned to Copilot.

Tip: start small. Inline first — escalate to Chat → Agent → Cloud only as the surface area grows.

/explain — what to feed it, what to expect back

Feed it

- A stack trace (paste into chat or use #terminalSelection).
- A specific function or selection — narrow beats broad.
- The relevant test output (failing assertion + actual/expected).
- A symbol with #usages so Copilot sees the call sites.
- An error message verbatim — do NOT paraphrase.
- A log snippet via #terminalLastCommand.
- #problems for the active file's lint / type diagnostics.

Expect back

- A walk-through of what the code does, function by function.
- Likely cause of the error — usually one strong candidate, sometimes 2-3 ranked.
- A reproduction recipe (inputs that trigger the path).
- Suggested next checks — print statements, debugger watches, test cases.
- A patch — only if you ask. /explain stays read-only by default.
- Confidence cues — "likely", "may", "if X then Y" — read them, do not skip them.

/fix — patterns for inline fixes

ERROR

```
/fix the null reference in the selected method
```

Pin the specific exception. Generic "/fix this" tends to over-rewrite.

TYPE

```
/fix the TypeScript "X is possibly undefined" on line N
```

Quote the diagnostic verbatim — Copilot is excellent at compiler/lint errors.

TEST

```
/fix the failing assertion in #selection — output #terminalSelection
```

Feed the failure output as evidence; otherwise Copilot guesses.

BEHAVIOUR

```
/fix #selection so that empty inputs return Result.NotFound
```

State the expected behaviour. Without it, /fix narrows the symptom, not the cause.

PERFORMANCE

```
/fix #selection — this loop is O(n²); reduce to O(n)
```

Pin the complexity goal. Otherwise /fix may "fix" by short-circuiting cases.

Failing tests — feed Copilot the failure, not the function

```prompt
// Bad prompt — Copilot guesses

/fix this test
-------------------------------------------------------------------------------
// Good prompt — Copilot reads evidence

The test in #selection fails with:
#terminalSelection

Trace through #file:OrderService.ts
to find the root cause, then propose
the smallest patch.

Do NOT modify the test until we
confirm the failure is in production
code, not the test itself.
```

Diagnostic order of operations

- Read the failure output. Do not paraphrase it to the AI.
- Confirm the test asserts the right thing — bad tests fail too.
- Ask Copilot to trace the code path that the failing case exercises.
- Propose a hypothesis. Add a print or watch to verify it.
- Patch. Re-run. If still red, the hypothesis was wrong — start over.
- Never delete a failing test "to make CI green".

In agent mode, this whole loop runs autonomously — Copilot reads the failure, edits the code, re-runs tests, and stops when green. You review the diff.

Memory leaks — sustained growth, not single allocations

Patterns Copilot recognises

- Listeners registered but never removed.
- Long-lived caches with no eviction.
- Closures retaining large objects.
- Timers / intervals never cleared.
- Static collections growing forever.
- Stream / DB cursor not disposed.
- Circular refs in native bindings.

prompt

```
// Prompt template

Heap dump summary (top retained):
#terminalSelection

Process: long-running worker.
Symptom: RSS grows 2MB / hour;
         restart needed every ~12h.

For each top-retained type:
 1. Trace WHO keeps it alive
    (use #usages on the type).
 2. Name the retention path.
 3. Suggest the minimum patch
    (dispose, weak-ref, cap, TTL).

No big-bang rewrites.
```

Log analysis — paste, group, ask for the story

prompt

```
// Prompt template

Below are 1500 lines of prod logs
around the 14:32 UTC outage.
#terminalSelection

Tasks:
 1. Group repeated events; show counts.
 2. Build a timeline of distinct events
    leading to the failure.
 3. Identify the first abnormal line
    and the most likely root cause.
 4. Suggest 3 follow-up queries to
    confirm the hypothesis.

Do NOT propose fixes yet — diagnose first.
```

Two patterns that work

Bulk paste + group + timeline.

For up to ~2000 lines, paste into chat and ask Copilot to group identical events and build a timeline. Token-efficient and surprisingly accurate.

MCP-backed observability.

Connect a Sentry / Datadog / Dynatrace MCP server. The agent queries the system live: "Show me errors for service X in the last 30m, grouped by stack-trace top frame."

Watch for: Copilot will confidently summarise hallucinated patterns from missing data. Always sanity-check the counts.

Visual Studio Debugger Agent (Apr 2026 GA)

What it does that Chat cannot

- Drives a real debug session — sets breakpoints, steps, inspects.
- Reads runtime values, not just static code.
- Asserts hypotheses against actual variable state.
- Iterates: hypothesis → breakpoint → observe → revise.

When to invoke it

- A bug only reproduces with a real input you cannot easily encode.
- State is the question — "what is this dict at line 87 the second time we hit it?"
- Static reasoning has exhausted its useful range.
- You want the AI to drive the same workflow a senior dev would.

Debugger Agent prompt

```
// Prompt template

Reproduction: open Orders.cs, run unit test "ChargeFailsAfterDecline" under the debugger.
On line 88, watch `customer.PaymentMethods[0]`.

1.  Step until the test fails.
2.  Report the runtime values that contradict my expectation that PaymentMethods is non-empty.
3.  Propose the smallest fix — and write a regression test for the empty-payment-methods case.
```

NEW · CLOUD AGENT

Bug from a GitHub issue → repro → PR — Cloud Coding Agent

1

Open issue

Title + bug + reproduction steps. Add labels.

2

Assign Copilot

Assignees → Copilot. Optional inline prompt.

3

Reproduce

Agent reads code, runs tests, narrows scope.

4

Patch + verify

Agent edits in a branch, runs CI, self-fixes.

5

Draft PR + review

@copilot in comments to iterate. You merge.

issue.md

```
// Sample issue body — small, evidence-rich issues work best

Repro: POST /orders with payment_methods: []   ->   500 NullReferenceException
Expected: 422 with "no payment method" error message.
Logs: see attached trace.   Affected file: src/api/Orders.cs.
Acceptance: failing regression test added + 422 returned + CHANGELOG entry.
```

A custom "Bug-Hunter" agent — scoped, hypothesis-driven

agent.md

```
# .github/agents/bug-hunter.agent.md
---
name: Bug-Hunter
description: Diagnoses bugs from evidence
tools: ['search/codebase','read/problems',
        '#fetch','#terminalLastCommand']
model: ['Claude Sonnet 4.6','GPT-5.2-Codex']
handoffs:
  - label: Apply Minimal Fix
    agent: agent
---

# Instructions
You may NOT edit any files.
For every bug report:
 1. Quote evidence (trace, log, test).
 2. Form 1-3 ranked hypotheses.
 3. Propose a verification step.
 4. Hand off the winning fix.
```

Why read-only is the point

A bug-hunter that can edit code is a junior developer with an itchy trigger finger. Locking the tool list to read + search forces the agent (and you) to identify the cause before applying any patch.

Workflow

- Bug-Hunter diagnoses → 1-3 ranked hypotheses.
- You pick the most likely, run the verification step.
- Click handoff → switches to agent mode, applies minimal fix.
- Re-run tests; if still red, return to Bug-Hunter.

Debugging — key takeaways

1

Reproduce first

A bug without a repro is a story. Without a failing test, your fix is a story too.

2

Feed evidence, not labels

Paste the exact trace, log, or test output. Paraphrasing degrades the AI signal.

3

Hypotheses, not answers

/explain returns ranked candidates. Verify with a test or a watch before patching.

4

Smallest fix wins

"What is the minimum patch?" beats letting Copilot rewrite half the file.

5

Use the right surface

Inline → Chat → Agent → Debugger Agent → Cloud — escalate only as scope grows.

6

Close the loop

Regression test, postmortem note, PR with the failing case captured. Drift is the enemy.

Part 3 · Refactoring

Modernising legacy code safely with #codebase, agent mode, and a scoped Refactorer agent.

GH-300 · Code Health with GitHub Copilot

Three refactoring strategies — match the strategy to the smell

Strategy 1

Hard to read

Understand current behaviour → simplify without changing it → run tests.

/explain #selection → "simplify #selection" → run tests.

Strategy 2

Duplication

Extract repeated logic into a reusable function → locate all duplicates → consolidate.

"Find duplicated logic across #file:user.js and #file:order.js."

Strategy 3

Outdated patterns

Modernise syntax + patterns → verify same behaviour → deploy.

"Convert to async/await" / "Use modern ES6+ syntax".

Five common refactoring tasks — and the right ask for each

Simplify conditionals
Convert nested if/else to guard clauses or early returns.

```
"Simplify #selection — the nested if/else is hard to follow."
```

Extract magic numbers
Replace hardcoded values with named constants.

```
"Extract magic numbers in #selection to named constants with descriptive names."
```

Break down large functions
Identify logical sections and propose focused helpers.

```
"This function is too long. Break it into smaller focused functions, keep behaviour identical."
```

Remove duplication
Analyse files, identify patterns, propose shared utilities.

```
"Find duplicated logic in #file:user.js and #file:order.js. Propose a shared utility."
```

Convert to async / await
Modernise async patterns, improve readability.

```
"Convert the callback-based code in #selection to async/await; keep error semantics."
```

Performance refactoring — profile first, prompt second

```prompt
// Profile-first prompt

Profile output (top 5 frames):
#terminalSelection

Source:  #file:reports.ts.
Propose the smallest change with the
biggest impact. Quantify the win.
```

Common optimisations Copilot suggests

- Replace O(n^2) with O(n) via hash maps.
- Memoise expensive computations.
- Batch API calls; cut round-trips.
- React: memo, useMemo, useCallback.

A note on /optimize and /simplify

These appear in older docs as built-in slash commands, but in current VS Code / Visual Studio / JetBrains they are not first-class slash commands — typing /optimize lands as a plain chat prompt. Treat them as natural-language asks ("optimise this for time complexity"), not as guaranteed commands. Use the slash-command list (type / in chat) to see what is actually wired up in your environment.

Always profile before optimising. Always run tests after. "Faster" code that breaks behaviour is worse than slow code.

WORKFLOW

Legacy modernisation — Understand → Test → Modernize → Verify

1

Understand current

/explain #selection — what does this legacy code do, what patterns does it use?

2

Generate tests

/tests #selection — lock in current behaviour BEFORE modernising anything.

3

Modernise incrementally

"Modernise #selection — convert to ES6+, use async/await, replace deprecated APIs." One concern per pass.

4

Verify

Run the tests from step 2. Green → behaviour preserved. Red → diagnose, do not "fix the test".

A custom "Refactorer" agent — scoped to non-test files

agent.md

```
# .github/agents/refactorer.agent.md
---
name: Refactorer
description: Refactor without touching tests
tools: ['search/codebase','editFiles',
        'read/problems','search/usages']
model: ['Claude Sonnet 4.6','GPT-5.2-Codex']
handoffs:
  - label: Run Tests
    agent: agent
    send: false
---

# Instructions
You are the Refactorer.

You may EDIT production files only.
You may NEVER edit any file matching:
  - **/*Tests.cs    - **/*_test.py
  - **/*.test.ts    - tests/**

Per turn: state the target smell;
show ONE refactor; hand off to run tests.
```

Why this enforcement matters

A refactoring agent that can edit tests can "make the tests pass" by deleting the assertions. Scoping its tools to production code only enforces the refactoring contract: behaviour is preserved, verified by tests it cannot change.

Pair with handoff

"Run Tests" handoff (send: false) switches to default agent mode to run the suite. The two-step rhythm — refactor → test — is now structural, not discipline-based.

Codebase-wide refactors — #codebase + agent mode

codebase-wide prompt

```
// Prompt template — agent mode

Find every call site of UserService.GetById that does not handle the new NotFoundResult return type.
Use #codebase to locate them, then update each call site to the new pattern.
For each file you change, run the relevant test class and report any failures.
Do NOT change the signature of GetById itself.
Stop and ask if more than 20 files need to change — that is a different conversation.
```

Where it shines

Type / API rename across 50+ files, deprecating a helper, propagating a new optional parameter, lifting a pattern out of every controller.

Use Plan mode first

For >20 files, let Plan mode propose the change graph before any edit. Review the plan; approve scoped phases.

Cap the blast radius

"Stop and ask if more than N files" — a sentence in the prompt that prevents a runaway agent.

Verify per slice

Run tests after each batch, not at the end. Smaller red signals = faster diagnosis.

Dead-code cleanup — the refactor nobody schedules

How to find it

- #usages to find functions with zero call sites.
- Coverage report — code never exercised.
- Feature-flag audit — flags long-since flipped.
- TODO / FIXME comments older than 18 months.

Watch for

- Reflection / dynamic dispatch — static usage analysis misses these.
- Public API for external consumers — "unused" internally, critical externally.
- Test-only entry points — may look unused but unblock tests.
- Removed code that lingers in error paths.

dead-code audit

```
// Cleanup prompt — agent mode

Use #usages to find every public function in #file:Utils.ts with zero call sites in the codebase.
For each, check: is it exported from index.ts (potentially a public API)?  Is it a test-only entry point?
Produce a list grouped as "safe to delete" / "needs review" / "keep" with one-line justification.
Do NOT delete anything; we will review the list together first.
```

Refactoring command reference — with footnotes

Command / prompt	When to use	Status
`/explain #selection`	Understand legacy code before modifying.	**Built-in**
`/tests #selection`	Lock behaviour before refactoring.	**Built-in**
`"Simplify #selection"`	Clean up complex logic, nested conditionals.	**Natural language**
`"Optimise #selection for ..."`	Quantified performance refactor.	**Natural language**
`"Extract magic numbers"`	Named constants for hardcoded values.	**Natural language**
`"Break into smaller functions"`	Decompose large functions.	**Natural language**
`"Convert to async/await"`	Modernise callback-based code.	**Natural language**
`"Find duplicated logic in #file"`	Identify and consolidate duplication.	**Natural language**

Type / in chat to confirm what is wired up in your environment. Natural-language items work everywhere; slash-prefixed items vary by IDE.

CLOSE

Refactoring — key takeaways

1

Three strategies, match the smell

Hard-to-read → simplify · Duplication → extract · Outdated → modernise. Each has its own workflow.

2

Tests first or do not refactor

Lock behaviour with /tests before touching code. Without tests, refactoring is wishful thinking.

3

One concern at a time

Simplify OR optimise OR modernise — not all three in one commit. Smaller diffs = traceable bugs.

4

Dead code is one prompt away

#usages finds functions with zero call sites; agent mode removes them and re-runs the suite to prove nothing broke.

5

Use a Refactorer custom agent

Tool scope locks the contract: edit production, never tests. Pair with a "Run Tests" handoff.

6

Scale with #codebase + agent

Codebase-wide renames and migrations are now a prompt + a cap on blast radius, not a multi-week project.

Thank you.

Bring a flaky test, a stubborn bug, and one ugly module — we will work all three.

Sandeep Soni · Deccansoft Software Services Pvt Ltd